

EventReservation

Documentação Técnica Completa

API REST em ASP.NET Core (com autenticação JWT e testes automatizados) + Frontend em React

Vitor Miranda Jeremias

Estudante de Engenharia de Software - PUCPR

github.com/Vitormjere

Sumário

- 1. Visão geral
- 2. Arquitetura geral
- 3. Backend - EventReservation API
- 4. Frontend - EventReservationFrontend
- 5. Deploy e CI/CD
- 6. Desafios reais e como foram resolvidos
- 7. Como rodar os projetos localmente

1. Visão geral

O EventReservation é um sistema full-stack para gerenciamento de eventos e reservas, composto por uma API REST em C# (backend) e uma aplicação em React (frontend), que se comunicam de ponta a ponta.

Desenvolvi esse projeto para fortalecer meu portfólio em áreas que ainda não tinha explorado tanto, principalmente autenticação e autorização com JWT, hashing de senhas, controle de permissões por papéis de usuário e testes automatizados, tanto unitários quanto de integração.

O sistema gerencia três entidades principais: usuários, eventos e reservas. Além disso, aplica regras de negócio que simulam um cenário real, como limite de participantes por evento, prevenção de reservas duplicadas, bloqueio de reservas em eventos que já aconteceram e controle para que cada usuário possa gerenciar apenas os recursos que lhe pertencem.

Links

API em produção: <https://eventreservation-api-vitor-dugnc2fra6d3hne7.brazilsouth-01.azurewebsites.net/scalar>

Frontend em produção: <https://event-reservation-frontend.vercel.app>

Repositório backend: <https://github.com/Vitormjere/EventReservation>

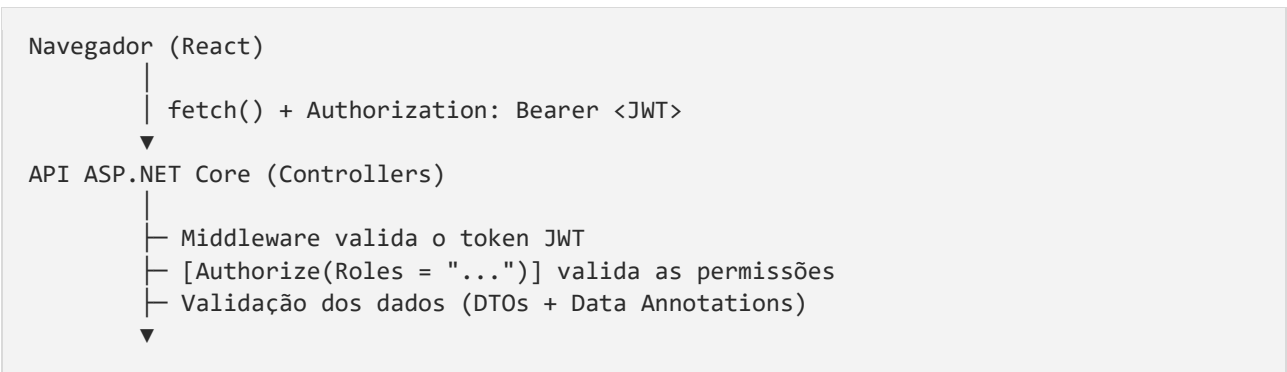
Repositório frontend: <https://github.com/Vitormjere/EventReservationFrontend>

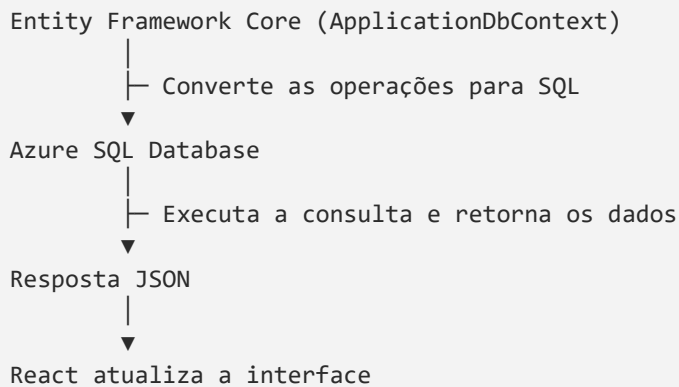
Stack utilizada

Camada	Tecnologias
Backend	ASP.NET Core 9, Entity Framework Core, SQL Server / Azure SQL, JWT, ASP.NET Core Identity (hashing)
Testes	xUnit (unitários e integração), SQLite em memória, WebApplicationFactory
Frontend	React (Vite), React Router, Context API, CSS puro
Infraestrutura	Azure App Service, Azure SQL Database, Vercel
CI/CD	GitHub Actions (backend), deploy automático Vercel (frontend)

2. Arquitetura geral

O fluxo de uma requisição autenticada típica segue este caminho:





O backend segue uma arquitetura em camadas, dividida em quatro projetos dentro da mesma solution: Domain (são entidades e regras de negócio puras, sem nenhuma dependência externa), Infrastructure (EF Core, DbContext, migrations), Api (Controllers, autenticação, configuração) e Tests (testes unitários e de integração).

EventReservation.Api	-> depende de Domain e Infrastructure
EventReservation.Infrastructure	-> depende de Domain
EventReservation.Domain	-> não depende de nada
EventReservation.Tests	-> depende de tudo

A principal ideia dessa arquitetura é manter a camada Domain independente de tecnologias como Entity Framework Core, ASP.NET Core ou qualquer outro detalhe de infraestrutura. Dessa forma, as regras de negócio ficam desacopladas da implementação técnica, tornando o código mais organizado, reutilizável e fácil de testar. Por exemplo, é possível validar a capacidade máxima de um evento sem precisar de um banco de dados ou de um servidor HTTP em execução.

No frontend, a aplicação é organizada em pages, que representam as telas da aplicação; components, que reúnem elementos reutilizáveis, como a Navbar e as rotas protegidas; context, responsável pelo gerenciamento global da autenticação; e api, que centraliza todas as requisições HTTP. A navegação entre as páginas é controlada pelo React Router, que decide qual tela exibir com base na URL acessada

3. Backend - EventReservation API

3.1 Modelagem de dados

O domínio é composto por três entidades principais: User, Event e Reservation, além dos enums UserRole e ReservationStatus. Um Event pertence a um usuário organizador, enquanto uma Reservation representa a reserva de um usuário participante em um determinado evento, armazenando também informações próprias, como o status da reserva (Confirmed ou Cancelled)

Uma decisão importante durante o desenvolvimento foi modelar Reservation como uma entidade própria, em vez de utilizar apenas uma tabela de junção entre User e Event. Isso porque a reserva possui dados que pertencem à relação entre usuário e evento, como o status e a data de criação, e não fazem parte diretamente de nenhuma das duas entidades

3.2 Decisões técnicas e por quê

Controllers em vez de Minimal APIs

Escolha: Utilização de Controllers (ApiController) para organizar os endpoints por recurso, como AuthController, EventController e ReservationController

Motivação: Optei pelos Controllers porque eles deixam a separação de responsabilidades mais clara e seguem o padrão mais utilizado em projetos profissionais. Além disso, essa abordagem facilita a organização de funcionalidades como autenticação, autorização e injeção de dependência, tornando o projeto mais estruturado e servindo como uma boa forma de consolidar esses conceitos

Alternativa considerada: *Minimal APIs: mais rápidas para projetos pequenos, porém menos estruturadas para um projeto com múltiplas camadas de autorização*

Autenticação com JWT + hashing de senha

Escolha: Implementação de autenticação baseada em tokens JWT, utilizando o algoritmo HS256 para assinatura. As senhas dos usuários nunca são armazenadas em texto puro, sendo protegidas por meio de hashing com PasswordHasher<User>, do ASP.NET Core Identity

Motivação: Optei pelo JWT porque ele permite uma autenticação stateless, ou seja, o servidor não precisa armazenar sessões em memória ou no banco de dados. A cada requisição, basta validar a assinatura do token para confirmar a identidade do usuário. Já o uso de hashing garante que as senhas não possam ser recuperadas, mesmo que alguém tenha acesso ao banco de dados, oferecendo uma camada importante de segurança

Alternativa considerada: *Autenticação via cookie de sessão: mais simples de implementar, porém exige estado no servidor e não se adapta bem a um frontend SPA consumindo a API de forma desacoplada*

Autorização por papel e por propriedade

Escolha: Utilização de duas camadas de autorização: atributos como [Authorize(Roles = "Organizer,Admin")] para controlar o acesso com base no papel do usuário e verificações no próprio código para validar a propriedade dos recursos

Motivação: Essa abordagem foi escolhida porque nem todas as regras de autorização podem ser resolvidas apenas com atributos. Enquanto a permissão para criar eventos pode ser controlada pelo papel do usuário, ações como editar ou excluir um evento exigem uma validação adicional para confirmar se ele realmente pertence ao usuário autenticado. Para isso, a aplicação busca o recurso, compara o identificador do proprietário com o ID presente nas claims do token JWT e só permite a operação caso ambos correspondam

Alternativa considerada: *Delegar toda a autorização para um serviço de políticas customizadas (IAuthorizationHandler): mais elegante em sistemas grandes, porém desnecessário para o escopo deste projeto*

Entity Framework Core (Code First) com Fluent API para relacionamentos

Escolha: Utilização da abordagem Code First, em que as classes C# definem o modelo da aplicação e o banco de dados é criado e atualizado por meio de Migrations. Os relacionamentos mais complexos foram configurados explicitamente com a Fluent API no método OnModelCreating

Motivação: Essa abordagem oferece maior controle sobre o modelo de dados e facilita a evolução do banco ao longo do desenvolvimento. Durante a implementação, havia mais de um caminho de relacionamento entre User e Reservation, o que fazia o comportamento padrão de exclusão em cascata do Entity Framework Core gerar um ciclo que o SQL Server não permitia criar. Para resolver esse problema, configurei DeleteBehavior.Restrict nas chaves estrangeiras que apontam para User. Além de eliminar o ciclo de dependências, essa configuração também evita que a exclusão de um usuário apague automaticamente seu histórico de eventos e reservas

Alternativa considerada: *Deixar o comportamento padrão (Cascade): mais simples, porém tecnicamente inviável neste modelo e perigoso do ponto de vista de integridade de dados*

DTOs para entrada e saída

Escolha: Utilização de DTOs para todas as entradas e saídas da API. Dessa forma, as entidades do Entity Framework Core não são expostas diretamente, sendo utilizadas classes específicas como RegisterRequest, EventResponse e ReservationResponse

Motivação: Essa decisão aumenta a segurança da aplicação, pois impede que informações sensíveis, como PasswordHash, sejam enviadas ao cliente. Também evita que o usuário consiga definir campos que devem ser controlados pela própria aplicação, como o identificador de um recurso. Além disso, o uso de DTOs desacopla o contrato público da API das entidades do domínio, permitindo que a estrutura interna evolua sem impactar os clientes que consomem a API

Alternativa considerada: *Expor as entidades diretamente: mais rápido de escrever, porém inseguro (vazaria o hash da senha) e propenso a problemas de serialização circular (User → Reservations → User...)*

Regras de negócio isoladas na entidade de domínio

Escolha: O método CanReserve, responsável por verificar se um usuário pode realizar a reserva de um evento, foi implementado na própria entidade Event, localizada na camada Domain, em vez de ficar em um serviço ou no Controller

Motivação: Essa decisão mantém a regra de negócio próxima da entidade à qual ela pertence, tornando o código mais organizado e coerente. Como o método utiliza apenas dados que já estão carregados em memória, como a lista de reservas do evento, ele não depende de banco de dados nem de outros componentes da aplicação. Isso permite criar testes unitários simples e rápidos, sem necessidade de mocks ou qualquer infraestrutura adicional

Alternativa considerada: *Colocar a validação diretamente no Controller: mais rápido de escrever, porém mistura regra de negócio com lógica de transporte HTTP, dificultando o teste isolado da regra*

Retry-on-failure na conexão com o banco

Escolha: Configuração do EnableRetryOnFailure no Entity Framework Core, permitindo até cinco tentativas de reconexão com atraso progressivo entre elas

Motivação: Durante o desenvolvimento e o deploy, identifiquei que o Azure SQL Database, utilizando o plano gratuito Serverless, pode entrar em estado de hibernação após um período sem uso. Quando isso acontece, a primeira tentativa de conexão pode falhar com erros como o 40613 ou por timeout. Com o EnableRetryOnFailure, a aplicação tenta se conectar novamente de forma automática, tornando o sistema mais resiliente e evitando que esse problema afete o usuário

Alternativa considerada: *Sem retry, qualquer chamada após período de inatividade do banco resultaria em erro 500 para o usuário, exigindo nova tentativa manual.*

Seed automático de usuário administrador

Escolha: Na inicialização da aplicação, é realizada uma verificação para identificar se já existe um usuário com o papel de Admin. Caso não exista, um administrador é criado automaticamente, com a senha armazenada de forma segura utilizando hashing

Motivação: O cadastro público da aplicação cria apenas usuários com o papel de Participant, uma decisão adotada para evitar a criação indevida de administradores. Por isso, foi necessário implementar um mecanismo de bootstrap para garantir a existência do primeiro usuário administrador. Além disso, essa rotina é envolvida por um try/catch, evitando que uma indisponibilidade temporária do banco de dados impeça a inicialização da aplicação.

Alternativa considerada: *Promoção manual direta no banco de dados: funciona, porém não é reproduzível automaticamente ao clonar o projeto do zero*

3.3 Exemplo de código - regra de negócio da reserva

Trecho da entidade Event, que concentra a validação principal do sistema:

```
public (bool CanReserve, string? ErrorMessage) CanReserve(
    int userId, DateTime currentDate)
{
    if (EventDate < currentDate)
        return (false, "Não é possível reservar um evento que já ocorreu.");

    var confirmedReservations = Reservations
        .Count(r => r.Status == ReservationStatus.Confirmed);

    if (confirmedReservations >= Capacity)
        return (false, "Este evento atingiu a capacidade máxima.");

    var alreadyReserved = Reservations.Any(r =>
        r.UserId == userId && r.Status == ReservationStatus.Confirmed);

    if (alreadyReserved)
        return (false, "Você já possui uma reserva confirmada.");

    return (true, null);
}
```

```
}
```

O método usa guard clauses (retorno antecipado a cada regra violada) em vez de condicionais aninhadas, e devolve um tuple nomeado contendo tanto o resultado booleano quanto a mensagem de erro específica, evitando a criação de uma classe adicional só para esse propósito.

3.4 Validação de dados

Os DTOs de entrada usam Data Annotations para validação automática. O atributo [ApiController] faz o ASP.NET Core validar o modelo antes mesmo do código do Controller executar, retornando 400 automaticamente quando os dados são inválidos.

```
public class RegisterRequest
{
    [Required, MaxLength(100)]
    public string Name { get; set; }

    [Required, EmailAddress]
    public string Email { get; set; }

    [Required, MinLength(6)]
    public string Password { get; set; }
}
```

3.5 Endpoints disponíveis

Recurso	Endpoints	Autorização
Auth	POST /api/Auth/register POST /api/Auth/login	Público
Events	GET /api/Events GET /api/Events/{id}	Público
Events	POST /api/Events PUT /api/Events/{id} DELETE /api/Events/{id}	Organizer / Admin (edição/exclusão exige ser o dono ou Admin)
Reservations	POST /api/Reservations GET /api/Reservations/my DELETE /api/Reservations/{id}	Qualquer usuário autenticado (cancelamento exige ser o dono ou Admin)

3.6 Testes automatizados

O projeto EventReservation.Tests cobre dois níveis distintos de teste, cada um com um propósito diferente:

Testes unitários

Testam lógica isolada, sem banco de dados nem servidor HTTP: o método CanReserve (cenários de capacidade esgotada, evento já ocorrido, reserva duplicada e sucesso) e o AuthService (hash de senha e verificação de senha correta/incorreta). Seguem o padrão AAA (Arrange, Act, Assert) e a convenção de nomenclatura Metodo_Cenario_ResultadoEsperado.

Testes de integração

Sobem a API inteira em memória usando WebApplicationFactory<Program>, fazendo requisições HTTP reais contra os Controllers, incluindo o pipeline completo de autenticação e autorização. Para isolar o

ambiente de teste do banco de produção, uma CustomWebApplicationFactory substitui o provider do Entity Framework Core (SQL Server) por SQLite em memória, garantindo testes rápidos, isolados e independentes da disponibilidade do Azure

```
public class CustomWebApplicationFactory : WebApplicationFactory<Program>
{
    protected override void ConfigureWebHost(IWebHostBuilder builder)
    {
        builder.ConfigureServices(services =>
        {
            // remove todo registro ligado ao provider original (SQL Server)
            // e registra o DbContext apontando para SQLite em memória
        });
    }
}
```

4. Frontend - EventReservationFrontend

4.1 Decisões técnicas e por quê

React com Vite

Escolha: Projeto criado com Vite

Por quê: Build e hot-reload significativamente mais rápidos que alternativas antigas, com configuração mínima necessária para começar

Context API para estado de autenticação

Escolha: Um AuthContext centraliza o usuário logado, o token e as funções login/logout, disponibilizados via hook customizado (useAuth) para qualquer componente da árvore

Por quê: Evita "prop drilling": repassar manualmente o estado de autenticação por múltiplos níveis de componentes que não usam esse dado diretamente, apenas o repassam adiante

Alternativa considerada: *Bibliotecas de gerenciamento de estado como Redux ou Zustand: desnecessárias para o volume de estado global deste projeto (apenas dados de sessão do usuário)*

Token JWT armazenado em localStorage

Escolha: O token retornado pela API é salvo em localStorage e reenviado automaticamente em cada requisição autenticada, via um cliente HTTP centralizado

Por quê: Simplicidade de implementação e persistência entre sessões (o usuário não precisa logar novamente a cada recarregamento de página). É uma escolha consciente com um trade-off de segurança conhecido: localStorage é acessível via JavaScript, o que o torna vulnerável a ataques de XSS, uma alternativa mais robusta seria um cookie httpOnly, que exigiria mudanças correspondentes na API (CORS com credentials, proteção CSRF)

Alternativa considerada: *Cookie httpOnly: mais seguro contra XSS, porém exige configuração adicional na API e não foi necessário para o escopo deste projeto de portfólio.*

Rotas protegidas por autenticação e papel

Escolha: Um componente `ProtectedRoute` envolve rotas sensíveis, redirecionando para o login se o usuário não estiver autenticado, ou para a home se o papel do usuário não estiver na lista de papéis permitidos

Por quê: Melhora a experiência do usuário, evitando que ele acesse (via URL direta) uma tela que não vai funcionar para seu papel. É importante notar que essa proteção é apenas de UX, a segurança real é garantida pela própria API, que valida autenticação e autorização de forma independente do frontend em cada requisição

CSS puro, sem framework

Escolha: Estilização manual, com variáveis CSS (custom properties) para cores, espaçamento e tipografia, formando um tema escuro consistente

Por quê: Opção deliberada de aprendizado: escrever CSS puro força o entendimento real de cada propriedade, sem depender de classes utilitárias prontas

Alternativa considerada: *Tailwind CSS, mais rápido para estilizar, porém esconde o comportamento do CSS por trás de classes utilitárias.*

fetch nativo em vez de Axios

Escolha: Toda comunicação HTTP usa a função `fetch`, nativa do navegador, encapsulada em um cliente central (`api/client.js`) que já injeta o header `Authorization` automaticamente.

Por quê: Para o volume de chamadas do projeto, os recursos extras do `Axios` (interceptors, cancelamento automático) não eram necessários, manter `fetch` simplifica o código e evita uma dependência a mais, além de reforçar o entendimento da API nativa da plataforma.

4.2 Padrão de comunicação com a API

Todas as chamadas HTTP são centralizadas em um único arquivo, `api/client.js`:

```
async function request(endpoint, options = {}) {
  const token = localStorage.getItem('token');
  const headers = { 'Content-Type': 'application/json', ...options.headers };
  if (token) headers['Authorization'] = `Bearer ${token}`;

  const response = await fetch(`${API_BASE_URL}${endpoint}`, { ...options, headers });

  if (!response.ok) {
    const errorData = await response.json().catch(() => null);
    throw new Error(errorData?.message || `Erro ${response.status}`);
  }
  return response.status === 204 ? null : response.json();
}
```

Centralizar essa função evita duplicar a lógica de autenticação e tratamento de erro em cada componente, e facilita trocar o ambiente (local ou produção) alterando uma única variável de ambiente (`VITE_API_URL`).

4.3 Formulários controlados

Cada campo de formulário é um input controlado: seu valor vem de uma variável de estado (useState), e cada alteração do usuário atualiza esse estado via onChange. Isso mantém a interface sempre sincronizada com os dados.

```
const [email, setEmail] = useState('');

<input
  value={email}
  onChange={(e) => setEmail(e.target.value)}
/>
```

4.4 Layout em sidebar

A navegação foi implementada como uma sidebar fixa lateral (flexbox), em vez de uma barra horizontal no topo, decisão tomada após revisão visual do layout inicial, que deixava muito espaço vazio em telas largas. A sidebar exibe links condicionalmente conforme o estado de autenticação e o papel do usuário logado.

5. Deploy e CI/CD

5.1 Backend - Azure App Service

A API está hospedada no Azure App Service (Linux, tier gratuito, reaproveitando o mesmo App Service Plan de outro projeto pessoal para economizar créditos), com banco de dados no Azure SQL Database (tier Serverless gratuito). O deploy é automatizado via GitHub Actions: a cada push na branch main, um workflow restaura, compila, publica especificamente o projeto da API (não a solution inteira, que também contém o projeto de testes) e implanta a aplicação automaticamente.

```
jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with: { dotnet-version: '9.0.x' }
      - run: dotnet restore
      - run: dotnet build --configuration Release --no-restore
      - run: dotnet publish EventReservation.Api/EventReservation.Api.csproj
        --configuration Release --no-build --output ./publish
      - uses: azure/webapps-deploy@v3
        with:
          app-name: 'eventreservation-api-vitor'
          publish-profile: '${{ secrets.AZURE_WEBAPP_PUBLISH_PROFILE }}'
          package: ./publish
```

A autenticação do workflow usa um Publish Profile (armazenado como Secret no GitHub), em vez de um Service Principal, necessário porque a conta institucional utilizada não possuía permissão de administrador no Azure Active Directory para gerar credenciais automaticamente. Todos os segredos de configuração (connection string, chave JWT) são armazenados como variáveis de ambiente do App Service, usando a convenção de nomenclatura Jwt__SecretKey (duplo underscore no lugar de dois-pontos, exigência de variáveis de ambiente do sistema operacional).

5.2 Frontend - Vercel

O frontend é hospedado como site estático na Vercel, também com deploy automático a cada push. A URL da API é injetada em tempo de build através da variável de ambiente `VITE_API_URL`, configurada diretamente no painel do projeto na Vercel, nunca commitada no repositório.

6. Desafios Reais e Como Foram Resolvidos

Esta seção documenta problemas reais enfrentados durante o desenvolvimento, úteis para revisão antes de entrevistas técnicas, pois mostram raciocínio de depuração aplicado a um projeto real, não apenas teoria.

Ciclo de cascade delete impedindo a criação do schema

Problema: Ao rodar a primeira migration, o SQL Server recusava criar as tabelas, pois existiam dois caminhos de cascade delete entre User e Reservation (via Event.Organizer e via Reservation.User).

Solução: Configuração explícita via Fluent API de `DeleteBehavior.Restrict` nas duas foreign keys que apontam para User, quebrando o ciclo e, como benefício adicional, protegendo o histórico de dados contra exclusão em cascata acidental.

Alterações no código não refletidas ao rodar comandos de terminal

Problema: Comandos como `dotnet build` e `dotnet ef` continuavam reportando erros de classes vazias (o conteúdo original do template), mesmo após o código correto ter sido escrito no editor.

Solução: Identificado que os arquivos tinham alterações não salvas no editor (indicado pelo asterisco na aba). O hábito de salvar tudo (`Ctrl+Shift+S`) antes de qualquer comando de terminal se tornou parte do fluxo de trabalho a partir desse ponto.

dotnet ef não conseguindo criar o DbContext em design-time

Problema: O comando `dotnet ef migrations add` falhava tanto ao tentar localizar o DbContext quanto, depois, ao tentar resolver o DbContextOptions necessário para instanciá-lo, porque a ferramenta não conseguia reaproveitar corretamente o host de injeção de dependência da aplicação (Minimal Hosting / top-level statements).

Solução: Implementação de uma classe `ApplicationDbContextFactory`, que implementa `IDesignTimeDbContextFactory<T>` e monta o DbContext manualmente para uso exclusivo de ferramentas de design-time, lendo a connection string diretamente do User Secrets via um GUID de `UserSecretsId`.

Banco de dados hibernado (erro 40613 / timeout de conexão)

Problema: O Azure SQL Database Serverless entra em hibernação após um período sem uso; a primeira query após esse período falhava com "Database is not currently available" ou timeout de conexão, tanto em ambiente local quanto em produção.

Solução: Configuração de `EnableRetryOnFailure` no EF Core, fazendo a aplicação tentar reconectar automaticamente (até 5 tentativas, com atraso progressivo) antes de reportar erro ao usuário.

Conflito de múltiplos providers de banco de dados nos testes de integração

Problema: Ao configurar a CustomWebApplicationFactory para usar SQLite em memória nos testes, a aplicação lançava InvalidOperationException informando que dois providers (SqlServer e Sqlite) estavam registrados simultaneamente no mesmo container de injeção de dependência.

Solução: Remoção explícita de todo serviço cujo namespace pertencesse a Microsoft.EntityFrameworkCore antes de registrar o novo provider SQLite, eliminando registros internos remanescentes do provider original.

CORS bloqueando o frontend em desenvolvimento e produção

Problema: O navegador bloqueava as requisições do React para a API por padrão, por serem origens diferentes, tanto em desenvolvimento (localhost:5173 vs localhost:7185) quanto após o deploy (domínio da Vercel vs domínio do Azure).

Solução: Configuração explícita de CORS no Program.cs, liberando tanto localhost:5173 (desenvolvimento) quanto o domínio de produção da Vercel, com atenção à ordem correta do middleware (UseCors antes de UseAuthentication).

Firewall do Azure SQL bloqueando conexões após mudança de IP

Problema: Erro "Client with IP address ... is not allowed to access the server" ao tentar conectar localmente, causado por uma mudança no IP público da conexão residencial, comum em provedores de internet dinâmicos.

Solução: Adição do novo IP às regras de firewall do servidor SQL do Azure diretamente pelo portal (função que detecta e sugere automaticamente o IP atual do cliente). Identificado como um problema recorrente a ser esperado durante o desenvolvimento local.

Falha ao gerar credenciais automáticas no GitHub Actions

Problema: O Visual Studio tentou criar uma Service Principal automaticamente para autenticar o deploy, mas falhou por falta de permissão de administrador na conta institucional (Azure AD).

Solução: Uso de Publish Profile como método alternativo de autenticação, com as credenciais de publicação básica habilitadas manualmente no App Service e o perfil configurado como Secret no repositório GitHub.

Variável de ambiente do frontend não aplicada em produção

Problema: Mesmo após o deploy na Vercel, o frontend em produção continuava tentando se conectar a https://localhost:7185, a URL de desenvolvimento, resultando em erro de CORS relacionado a endereços de loopback.

Solução: Identificado que a variável VITE_API_URL não havia sido salva corretamente durante a configuração inicial do projeto na Vercel. Como o Vite injeta variáveis de ambiente em tempo de build (não em runtime), foi necessário adicionar a variável nas configurações do projeto e forçar um novo deploy (Redeploy) para que o valor correto fosse embutido no bundle final.

7. Como Rodar os Projetos Localmente

7.1 Backend

```
git clone https://github.com/Vitormjere/EventReservation.git
cd EventReservation

dotnet user-secrets set "ConnectionStrings:DefaultConnection" "..." --project
EventReservation.Api
dotnet user-secrets set "Jwt:SecretKey" "..." --project EventReservation.Api
dotnet user-secrets set "Jwt:Issuer" "EventReservationApi" --project
EventReservation.Api
dotnet user-secrets set "Jwt:Audience" "EventReservationApiUsers" --project
EventReservation.Api

dotnet ef database update --project EventReservation.Infrastructure --startup-project
EventReservation.Api
dotnet run --project EventReservation.Api
```

Requer .NET 9 SDK, uma instância de SQL Server (local ou Azure SQL) e a ferramenta dotnet-ef instalada globalmente. Na primeira execução, um usuário administrador é criado automaticamente (admin@eventreservation.com / Admin@123).

7.2 Testes

```
dotnet test
```

Executa os testes unitários (regras de negócio e serviços) e de integração (fluxo HTTP completo com banco SQLite em memória) definidos no projeto EventReservation.Tests.

7.3 Frontend

```
git clone https://github.com/Vitormjere/EventReservationFrontend.git
cd EventReservationFrontend
npm install

# criar .env.local na raiz do projeto:
# VITE_API_URL=https://localhost:7185

npm run dev
```

Requer Node.js e a API do backend acessível (local ou remota)